

Project 1 - ScroogeCoin

Due: Sun Sep 27, 2026 11:59pm

Missing

10 Points Possible

Attempt 1



In Progress

NEXT UP: Submit Assignment



Add Comment

Unlimited Attempts Allowed

Available: Sep 7, 2026 12:00am until Sep 29, 2026 11:59pm

Details

CPSC 459 Blockchain Technologies

Project 1 – ScroogeCoin

Mode: Up to 4 persons in a team.

Register your team or look for a team HERE (https://docs.google.com/spreadsheets/d/1lxYpykev8SufN_GEOi3BzE7t119f-DxxEmqd1RJ-qck/edit?usp=sharing)

Due Date: as shown

Download Starter code here (<https://csufullerton.instructure.com/courses/3586477/files/284230557?wrap=1>) 
(https://csufullerton.instructure.com/courses/3586477/files/284230557/download?download_frd=1) !

Note that starter code does NOT run itself. The `main()` function is located at `TestTxHandler.java` which is provided to you as an autograder.

In ScroogeCoin, the central authority Scrooge receives transactions from users. You will implement the logic used by Scrooge to process transactions and produce the ledger. Scrooge organizes transactions into time periods or blocks. In each block, Scrooge will receive a list of transactions, validate the transactions he receives, and publish a list of validated transactions.

Note that a transaction can reference another in the same block. Also, among the transactions received by Scrooge in a single block, more than one transaction may spend the same output. This would of course be a double-spend, and hence invalid. This means that transactions can't be validated in isolation; it is a tricky problem to choose a subset of transactions that are together valid.

You will be provided with a Transaction class that represents a ScroogeCoin transaction and has inner classes Transaction.Output and Transaction.Input.

A transaction output consists of a value and a public key to which it is being paid. For the public keys, we use the built-in Java **PublicKey** class.

A transaction input consists of the hash of the transaction that contains the corresponding output, the index of this output in that transaction (indices are simply integers starting from 0), and a digital signature. For the input to be valid, the signature it contains must be a valid signature over the current transaction with the public key in the spent output.

More specifically, the raw data that is signed is obtained from the `getRawDataToSign(int index)` method. **Hint: you will need this function to be used as the first argument for `verifySignature(byte[] message, byte[] signature)`.**

You will be provided with an `rsa.jar` file for using the `RSAKey` class. The public key in a transaction output and the private key used to create signatures in transaction inputs are both represented by an `RSAKey`, and an `RSAKeyPair` is a public/private key pair.

The `RSAKey` class API is as follows:

```
public class RSAKey {  
  
    public RSAKey(BigInteger theExponent, BigInteger theModulus)  
  
    public BigInteger getExponent()  
  
    public BigInteger getModulus()  
  
    public byte[] encrypt(byte[] plaintext)  
  
    public byte[] decrypt(byte[] ciphertext)  
  
    public byte[] sign(byte[] message)  
  
    public boolean verifySignature(byte[] message, byte[] signature)  
  
    public int maxPlaintextLength()  
  
}
```

The `verifySignature()` method should be used by a public `RSAKey` object to verify a signature generated by the corresponding private `RSAKey`'s `sign()` method.

Note that you are only given code to verify signatures, and this is all that you will need for this assignment. The computation of signatures is done outside the `Transaction` class by an entity that knows the appropriate private keys.

A transaction consists of a list of inputs, a list of outputs and a unique ID (see the `getRawTx()` method). The class also contains methods to add and remove an input, add an output, compute digests to sign/hash, add a signature to an input, and compute and store the hash of the transaction once all inputs/outputs/signatures have been added.

You will also be provided with a UTXO class that represents an unspent transaction output. A UTXO contains the hash of the transaction from which it originates as well as its index within that transaction. We have included equals, hashCode, and compareTo functions in UTXO that allow the testing of equality and comparison between two UTXOs based on their indices and the contents of their txHash arrays.

Further, you will be provided with a UTXOPool class that represents the current set of outstanding UTXOs and contains a map from each UTXO to its corresponding transaction output. This class contains constructors to create a new empty UTXOPool or a copy of a given UTXOPool, and methods to add and remove UTXOs from the pool, get the output corresponding to a given UTXO, check if a UTXO is in the pool, and get a list of all UTXOs in the pool.

You will be provided with a file called **TxHandler.java** that implements the following API:

Your implementation of `handleTx()` should return **a mutually valid transaction set of maximal size** (one that can't be enlarged simply by adding more transactions). It need **NOT** compute a set of maximum size (one for which there is no larger mutually valid transaction set).

Based on the transactions it has chosen to accept, `handleTx` should also update its internal UTXOPool to reflect the current set of unspent transaction outputs, so that future calls to `handleTx()` and `isValidTx()` are able to correctly process/validate transactions that claim outputs from transactions that were accepted in a previous call to `handleTx()`.

Extra Credit: Create a second file called `MaxFeeTxHandler.java` whose `handleTx()` method finds a set of transactions with maximum total transaction fees -- i.e. maximize the sum over all transactions in the set of (sum of input values - sum of output values).

Academic Honesty:

All forms of cheating shall be treated with utmost seriousness. You may discuss the problems with other students, however, you must write your **OWN codes and solutions**. Discussing solutions to the problem is **NOT** acceptable (unless specified otherwise). Copying an assignment from another student or allowing another student to copy your work **may lead to an automatic F for this course**. Moss shall be used to detect plagiarism in programming assignments. If you have any questions about whether an act of collaboration may be treated as academic dishonesty, please consult the instructor before you collaborate. Details posted at <http://www.fullerton.edu/senate/documents/PDF/300/UPS300-021.pdf> (https://www.fullerton.edu/senate/publications_policies_resolutions/ups/UPS%20300/UPS%20300.021.pdf).

Autograder: **Download autograder here** (<https://csufullerton.instructure.com/courses/3586477/files/284230547?wrap=1>) 
(https://csufullerton.instructure.com/courses/3586477/files/284230547/download?download_frd=1)

An autograder is provided to you to self check the correctness of your program. Follow the following steps:

Step 1. Copy TxHandler.java into **grading** folder.

Step 2. While holding down **Windows key** press **R** (or click **Start**) and then type **cmd**. Press **Enter**.

Step 3. Move into the folder that contains the **ScroogeCoin**-related classes by typing this command:

```
cd C:\grading
```

Step 4. Compile the classes with this command (**Replace ; with : for Linux or Mac OS**):

```
javac -cp scroogeCoinGrader.jar;rsa.jar;algs4.jar;. TestTxHandler.java
```

Step 5. Finally, run the tests with this command:

```
java -cp scroogeCoinGrader.jar;rsa.jar;algs4.jar;. TestTxHandler
```

There are 15 tests in total and the test files are located in *files* folder.

Example output:

Test 1: test isValidTx() with valid transactions

==> passed

...

Testing MaxFeeTxHandler.java (extra credit):

Enter the following commands in your terminal from the current working directory (**Replace ; with : for Linux or Mac OS**).

```
javac -cp scroogeCoinGrader.jar;rsa.jar;algs4.jar;. TestMaxFeeTxHandler.java
```

```
java -cp scroogeCoinGrader.jar;rsa.jar;algs4.jar;. TestMaxFeeTxHandler
```

Note: If you are using IDE like eclipse, just add jar files to your build path and create a folder for all text files within the working directory of your code.

To import jar file in your Eclipse IDE, follow the steps given below

(<https://csufullerton.instructure.com/courses/3586477/pages/add-jar-to-eclipse-project>).

1. Right click on your project
2. Select Build Path
3. Click on Configure Build Path
4. Click on Libraries and select Add External JARs
5. Select the jar file from the required folder
6. Click and Apply and Ok

Jar Files:

- 1) rsa.jar: Contains classes for using RSAKeys
- 2) algs4.jar: Contains some useful classes like defining priority queues, stacks, etc.
- 3) scroogeCoinGrader.java: Contains classes used for grading the submitted files.

Grading for this assignment:

- Extra credit: 1 pt
- Late submissions less than 24 hours shall be penalized 10%.
- Late submissions greater than 24 hours shall be penalized 20%.
- No assignments shall be accepted after 48 hours.

Deliverables:

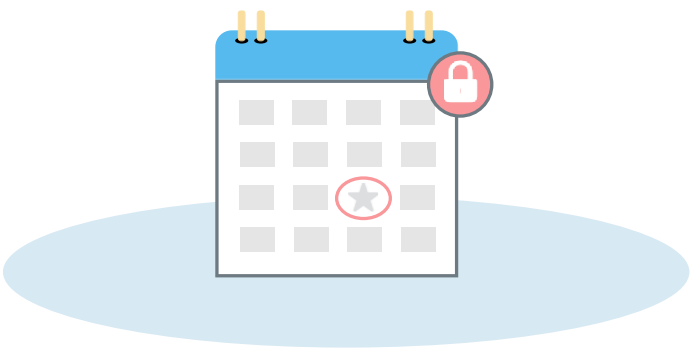
- Please hand in your source code electronically
- You must make sure that the code compiles and runs correctly.
- Write a README file which contains
 - Your Section#, Name and Email Address.
 - Whether you implemented the extra credit.
 - Anything special about your submission that we should take note of.
- **Only submit** TxHandler.java and README files.
- Submit MaxFeeTxHandler.java if you attempted extra credit work
- **One team only submit ONE copy.** Pick a team member who will be responsible for submitting the project. Other members do not need to submit. Canvas may mark your project as "Missing", but you do not need to worry about it. I'll contact you if I cannot find your name in any readme files.

▼ **View Rubric**

Project 1 Rubric		
Criteria	Ratings	Points

Test 1 view longer description	Full Marks autograder test 1 passed 0.6 pts	No Marks autograder test 1 not passed 0 pts	/0.6 pts
Test 2 view longer description	Full Marks autograder test 2 passed 0.6 pts	No Marks autograder test 2 not passed 0 pts	/0.6 pts
Test 3 view longer description	Full Marks autograder test 3 passed 0.6 pts	No Marks autograder test 3 not passed 0 pts	/0.6 pts
Test 4 view longer description	Full Marks autograder test 4 passed 0.6 pts	No Marks autograder test 4 not passed 0 pts	/0.6 pts
Test 5 view longer description	Full Marks autograder test 5 passed 0.6 pts	No Marks autograder test 5 not passed 0 pts	/0.6 pts
Test 6 view longer description	Full Marks autograder test 6 passed 0.6 pts	No Marks autograder test 6 not passed 0 pts	/0.6 pts
Test 7 view longer description	Full Marks autograder test 7 passed 0.6 pts	No Marks autograder test 7 not passed 0 pts	/0.6 pts
Test 8 view longer description	Full Marks autograder test 8 passed 0.7 pts	No Marks autograder test 8 not passed 0 pts	/0.7 pts
Test 9 view longer description	Full Marks autograder test 9 passed 0.7 pts	No Marks autograder test 9 not passed 0 pts	/0.7 pts

Test 10 view longer description	Full Marks autograder test 10 passed 0.7 pts	No Marks autograder test 10 not passed 0 pts	/0.7 pts
Test 11 view longer description	Full Marks autograder test 11 passed 0.7 pts	No Marks autograder test 11 not passed 0 pts	/0.7 pts
Test 12 view longer description	Full Marks autograder test 12 passed 0.7 pts	No Marks autograder test 12 not passed 0 pts	/0.7 pts
Test 13 view longer description	Full Marks autograder test 13 passed 0.7 pts	No Marks autograder test 13 not passed 0 pts	/0.7 pts
Test 14 view longer description	Full Marks autograder test 14 passed 0.8 pts	No Marks autograder test 14 not passed 0 pts	/0.8 pts
Test 15 view longer description	Full Marks autograder test 15 passed 0.8 pts	No Marks autograder test 15 not passed 0 pts	/0.8 pts
Extra credit test view longer description	Full Marks autograder MaxFee test passed 1 pts	No Marks autograder MaxFee test not passed 0 pts	/1 pts



Availability Dates
Sep 7 to Sep 29

[Previous](#)

[Next](#)

<https://csufullerton.instructure.com/courses/3586477/modules/items/92341875> <https://csufullerton.instructure.com/courses/3586477/modules/i>